

Basi di Dati

[Appunti di Piai Luca]

Academic Year 2023-2024

Indice

1	Metodologie e modelli per il progetto	5
1.1	Introduzione alla progettazione	5
1.1.1	Il ciclo di vita dei sistemi informativi	5
1.1.2	Metodologie di progettazione e basi di dati	6
1.2	Il modello Entità Associazione	6
1.2.1	Costrutti del modello	7
1.2.2	Rappresentazione estensionale dello schema ER	11
2	La progettazione concettuale	13
2.1	Pattern di progetto	13
3	La progettazione logica	17
3.1	Fasi della progettazione logica	17
3.2	Modello relazionale: definizioni	17
3.3	Ristrutturazione di schemi ER	19
3.3.1	Eliminazione delle generalizzazioni	20
3.4	Traduzione verso il modello relazionale	22
3.4.1	Entità e associazioni molti a molti	23
3.4.2	Associazione uno a molti	24
3.4.3	Il caso ricorsivo	25
3.4.4	Caso delle entità deboli	25
3.4.5	Caso in cui bisogna scegliere	26
4	Algebra e calcolo relazionale	27
4.1	Operazioni in algebra relazionale	27
4.1.1	Operazione di selezione	27
4.1.2	Operazione di proiezione	27
4.1.3	Operazione di ridenominazione	28
4.1.4	Operazioni insiemistiche: unione, intersezione e differenza	28
4.1.5	Operazione di correlazione: Join	29
4.1.6	Operazioni di raggruppamento	29
4.2	Algebra relazionale e valori nulli	30
5	Normalizzazione	32
5.1	Introduzione	32
5.2	La prima forma normale	32
5.3	La seconda forma normale	32
5.3.1	Terza forma normale e forma normale di Boyce e Codd	32

6	Generalità sul linguaggio SQL	34
6.1	La definizione delle tabelle	34
6.2	I comandi per la manipolazione dei dati	34
6.3	Il comando SELECT	35
6.4	Le operazioni relazionali nel linguaggio SQL	35
6.4.1	Operazione relazionale di selezione	35
6.4.2	Operazione relazionale di proiezione	35
6.4.3	Operazione relazionale di congiunzione	35
6.5	Le funzioni di aggregazione	36
6.5.1	Count	36
6.5.2	Sum	36
6.5.3	Avg	36
6.5.4	Min e max	36
6.6	Ordinamento	36
6.7	Raggruppamenti	37
6.8	Condizioni di ricerca	37
6.8.1	Between	37
6.8.2	In	37
6.8.3	Like	37

Capitolo 1

Metodologie e modelli per il progetto

1.1 Introduzione alla progettazione

1.1.1 Il ciclo di vita dei sistemi informativi

Esistono diversi metodi per gestire la progettazione di una base di dati. In generale non esiste un unico approccio, dipende da diversi fattori tra cui anche il tipo di base di dati che si vuole progettare. Nel nostro caso andremo a progettare dei database chiamati **database relazionali** (rimandiamo la definizione più avanti) e il modello che risulta più efficiente seguire è il **modello a cascata** (o waterfall).

Analizziamo brevemente i vari punti:

- **Studio fattibilità:** definisce i vincoli (denaro, tempo, limiti fisici, etc.).
- **Raccolta e analisi dei requisiti:** fase in cui vengono stilati i requisiti dettati dal committente attraverso interviste. Viene prodotta una lista il più dettagliata possibile, scritta in maniera informale.
- **Progettazione:** fase in cui vengono tradotti i requisiti della fase precedente in un linguaggio tecnico, solitamente vengono sfruttati modelli grafici. Nel nostro caso utilizzeremo il **linguaggio ER**.
- **Implementazione:** sfruttando i modelli prodotti dalla fase precedente, si realizza la base di dati. In questa fase utilizzeremo il linguaggio **SQL**.
- **Validazione:** viene prodotta una versione beta del software sui cui vengono effettuati i test che verificano i requisiti imposti dal committente.
- **Funzionamento:** il software prodotto viene mandato in produzione e si attuano attività finalizzate alla manutenzione del sistema.

Il processo non è mai sequenziale, durante l'esecuzione di una fase capita spesso di dover tornare alla fase precedente per rivedere alcune decisioni prese. Tuttavia è necessario far notare che il "salto indietro" non è gratuito, ha un costo in termini di tempo e denaro. Per evitare di incorrere in questo tipo di problemi è fondamentale valutare con attenzione le decisioni prese in una fase, prima di passare a quella successiva.

Nel corso ci concentreremo principalmente su tre fasi: requisiti; progettazione; implementazione.

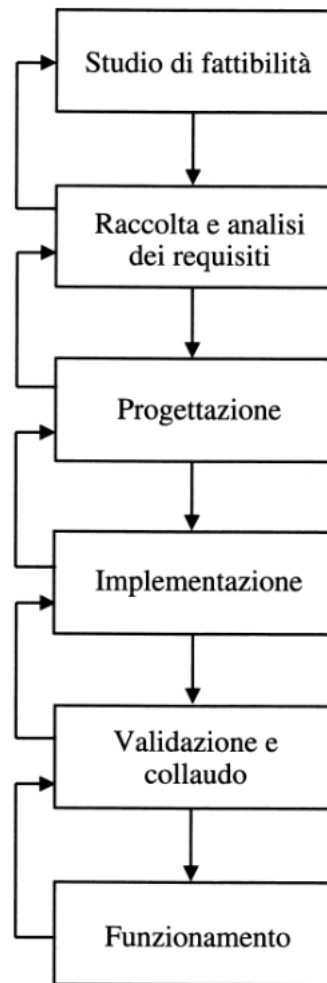


Figura 1.1: Modello a cascata

1.1.2 Metodologie di progettazione e basi di dati

La metodologia che andremo ad utilizzare è composta da tre fasi che si susseguono in cascata.

- **Progettazione concettuale.** Lo scopo è quello di rappresentare le specifiche informali della realtà di interesse in termini di una descrizione formale e completa. Da questa fase viene prodotto uno *schema concettuale* della base di dati, per produrlo utilizzeremo il *modello E-R*.
- **Progettazione logica.** In questa fase si va a tradurre lo schema concettuale della fase precedente in termini del modello di rappresentazione dei dati adottato dal sistema di gestione di basi di dati. Il prodotto di questa fase viene chiamato *schema logico* della base di dati.
- **Progettazione fisica.** Nell'ultima fase, lo schema logico, viene completato con la specifica dei parametri fisici di memorizzazione dei dati (quindi organizzazione file e indici). In questa fase viene prodotto lo *schema fisico*.

1.2 Il modello Entità Associazione

Il modello **modello E-R** è un modello concettuale che, attraverso i costrutti che offre, ci permette di schematizzare (attraverso un diagramma) l'organizzazione e la struttura delle

occorrenze dei dati, ovvero, dei valori assunti dai dati al variare del tempo. La figura 1.2 mostra tutti i costrutti che possiamo utilizzare.

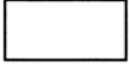
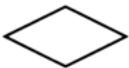
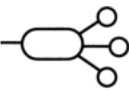
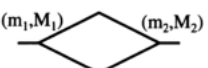
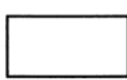
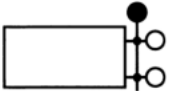
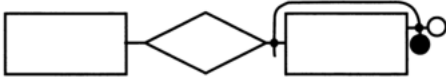
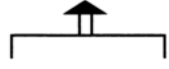
Costrutti	Rappresentazione grafica
Entità	
Relazione	
Attributo semplice	
Attributo composto	
Cardinalità di relazione	
Cardinalità di attributo	
Identificatore interno	 
Identificatore esterno	
Generalizzazione	
Sottoinsieme	

Figura 1.2: I costrutti del modello E-R

1.2.1 Costrutti del modello

Entità Rappresentano classi di oggetti (es: fatti, cose, persone) che hanno proprietà comuni ed esistenza "autonoma" ai fini dell'applicazione di interesse.

Associazioni Rappresentano legami logici, significativi per l'applicazione di interesse, tra due o più entità. Negli schemi E-R ogni associazione ha un nome che la identifica univocamente. Nota che due stesse entità possono avere più associazioni e un'associazione può essere *n*-aria, cioè può coinvolgere *n* entità.

Esistono anche le **associazioni ricorsive**. Nell'esempio di figura 1.3, l'associazione COLLEGA sull'entità IMPIEGATO connette coppie di impiegati che lavorano insieme, mentre l'associazione SUCCESSIONE sull'entità SOVRANO associa a ogni sovrano di una dinastia il suo successore. Nell'esempio, la prima associazione è simmetrica, la seconda non lo è. Se una relazione

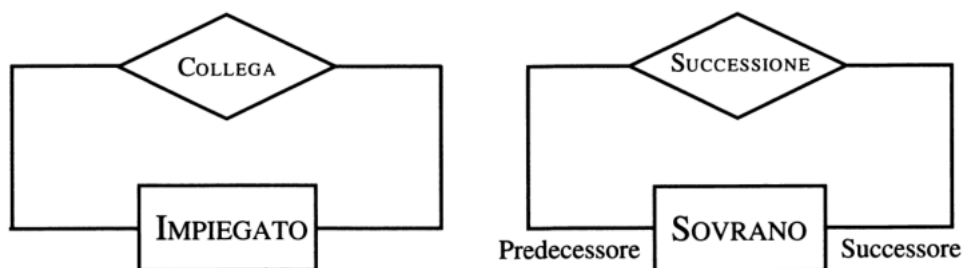


Figura 1.3: Esempi di associazioni ricorsive

ricorsiva non è simmetrica allora dobbiamo stabilire i *ruoli* che l'entità coinvolta gioca nella relazione. In base ai ruoli e alle cardinalità delle associazioni possiamo creare delle strutture ad albero o dei grafi anche usando dei database relazionali. Le prestazioni che si ottengono creando strutture gerarchiche di questo tipo in database relazionali non sono buone (causate dalle molteplici ricerche) e per questo motivo vengono utilizzati database a grafo che invece sono ottimi per questo tipo di utilizzi.

Attributi Descrivono le proprietà elementari di entità che sono di interesse ai fini dell'applicazione (es: COGNOME, NOME, ETA per l'entità PERSONA). Attributi affini possono anche essere raggruppati in insiemi, in questo modo andremo a formare un **attributo composto** (es: INDIRIZZO composto da VIA, CAP, CIVICO sempre per l'entità PERSONA). In generale gli attributi composti generano complicazioni, conviene utilizzare **attributi semplici**.

Esistono anche gli attributi delle associazioni. Vengono utilizzati quando l'attributo coinvolge tutte le entità che partecipano all'associazione.



Figura 1.4: Schema E-R con un'entità, attributi e un attributo composto

Cardinalità delle relazioni Le cardinalità vengono specificate per ogni relazione presente nel modello, esse indicano il numero minimo e massimo di istanze di associazione a cui un'istanza dell'entità può partecipare. In altre parole dicono quante volte, in una relazione tra entità, un'istanza di una di queste entità può essere legata a occorrenze delle altre entità coinvolte.

Nella figura stiamo indicando che ciascuna istanza di IMPIEGATO deve essere assegnato al minimo un incarico e al massimo 50. Viceversa ogni istanza di INCARICO deve essere assegnato al minimo nessun impiegato e al massimo 50. Ciò implica che non possono esistere impiegati a cui non sono stati assegnati incarichi ma possono esistere incarichi che non sono ancora stati assegnati a nessun impiegato.

Nella maggior parte dei casi, è sufficiente utilizzare tre valori: zero, uno e il simbolo *N* (indica un qualsiasi numero intero maggiore di uno). Inoltre precisiamo che:



Figura 1.5: Cardinalità di una relazione

- indicare cardinalità minima uguale a uno significa che la partecipazione dell'entità relativa è *obbligatoria*, se invece il minimo vale zero, la partecipazione è *facoltativa*;
- indicare cardinalità massima uguale a uno: la partecipazione dell'entità relativa può essere vista come una funzione (parziale se la cardinalità minima vale zero) che associa a un'istanza dell'entità una sola istanza (o nessuna) dell'altra entità che partecipa alla relazione. Se la cardinalità massima ha valore N allora esiste un'associazione con un numero arbitrario di istanze dell'altra entità.

Cardinalità degli attributi Nella maggior parte dei casi gli attributi hanno cardinalità $(1, 1)$, infatti se non viene specificato diversamente viene preso come valore di default. La cardinalità $(1, 1)$ sta ad indicare che ogni istanza di entità deve avere uno e un solo valore di tale attributo. Se vogliamo che tale attributo possa assumere il valore nullo, allora il valore di cardinalità minima deve essere uguale a zero.

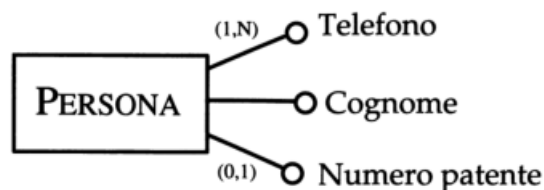


Figura 1.6: Cardinalità di un attributo

Anche per le cardinalità degli attributi se il valore minimo di cardinalità è pari a zero allora l'attributo è *facoltativo* mentre se vale uno allora è *obbligatorio*. Se il valore massimo di cardinalità è pari a N allora l'attributo è *multivalore*, se invece vale uno allora è a valore singolo.

Identificatori delle entità Si tratta di un attributo (o più) o di un gruppo di attributi che ci permettono di individuare univocamente un'istanza della relativa entità. Questi prendono il nome di **identificatori interni**. Tali attributi (per ovvie ragioni) devono avere cardinalità $(1, 1)$. Per ogni entità del nostro sistema dobbiamo individuare gli attributi che fanno parte di questo gruppo (almeno uno). Se non riusciamo ad individuare il gruppo degli identificatori, allora dobbiamo considerare l'idea di aggiungere un nuovo attributo all'entità presa in considerazione.

Nell'immagine, abbiamo scelto l'attributo TARGA come unico identificatore per l'entità AUTOMOBILE; infatti non possono esistere due automobili che hanno la stessa targa. Mentre per l'entità PERSONA abbiamo preso come identificatori gli attributi NOME, COGNOME, DATA DI NASCITA: abbiamo assunto che nella nostra applicazione non possano esistere due persone che abbiano tutti e tre i campi uguali. Quindi è chiaro che individuare gli attributi dipende molto dall'applicazione: non sempre vengono scelti gli stessi attributi come identificatori per una stessa entità in applicazioni diverse.

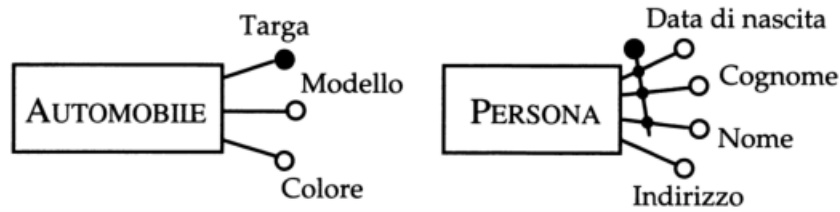


Figura 1.7: Identificatori interni

C'è anche da considerare che la scelta degli identificatori può influire sulle prestazioni della nostra base di dati. Infatti, ogni volta che viene inserita una nuova istanza di un'entità, bisogna assicurarsi che l'identificatore non sia già presente. Viene dunque sfruttato un algoritmo di ricerca per portare a termine il compito. Va da sé che più è semplice l'identificatore (es: ricercare un numero in una sequenza ordinata è efficiente) più risulta semplice la ricerca (es: le stringhe potrebbero richiedere degli algoritmi di pattern matching che potrebbero risultare molto costosi).

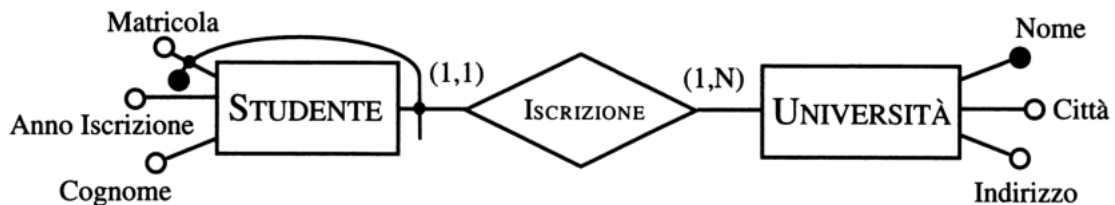


Figura 1.8: Identificatore esterno

Alcune volte gli attributi di un'entità non sono sufficienti ad identificare univocamente un'istanza. Nell'esempio riportato in figura, il solo attributo MATRICOLA non basta a differenziare tutti gli studenti, stiamo infatti registrando studenti appartenenti a diverse università, non è ragionevole supporre che abbiamo tutte matricole diverse. La soluzione più semplice è quella di identificare gli studenti attraverso la coppia MATRICOLA e UNIVERSITÀ. Nota che ciò è possibile grazie al fatto che ogni studente presente nel database deve essere iscritto ad una e una sola università, se STUDENTE partecipasse all'associazione con una cardinalità diversa da (1, 1), allora non potevamo usare UNIVERSITÀ come identificatore.

Nei casi in cui l'identificazione di un'entità è ottenuta utilizzando altre entità si parla di **identificatore esterno** e tale entità viene chiamata **entità debole**.

Generalizzazioni Rappresentano legami logici tra un'entità E , detta **entità genitore**, e una o più entità $E_1, \dots, E_n$ dette **entità figlie**, di cui E è più generale. Diremo che E è una generalizzazione di $E_1, \dots, E_n$ e che queste ultime sono specializzazioni di E . In questo modo andiamo in sostanza a creare dei sottoinsiemi all'interno dell'insieme delle istanze di E .

Le entità figlie ereditano tutti gli attributi del genitore, inoltre possono introdurre di nuovi validi solo per quella specializzazione.

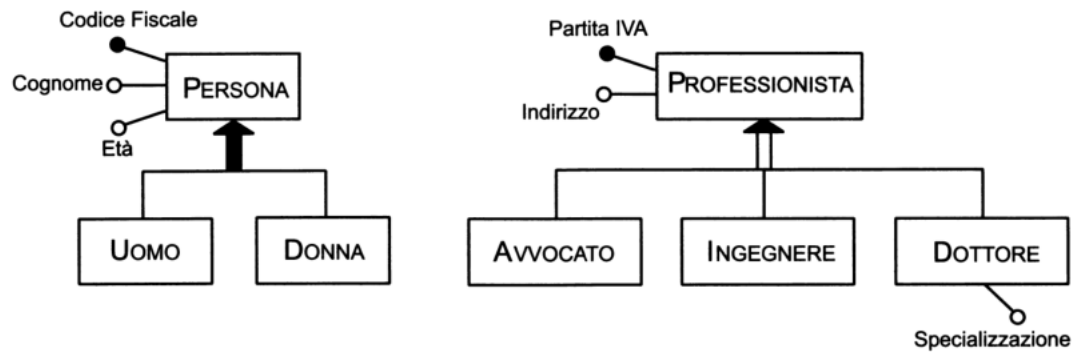


Figura 1.9: Esempi di generalizzazioni tra entit 

1.2.2 Rappresentazione estensionale dello schema ER

Si tratta di una visione insiemistica che permette di visualizzare le istanze delle entit  coinvolte e delle associazioni che le legano. Questi grafici sono utili per associazioni ternarie o pi , aiutano a verificare la correttezza del modello E-R.

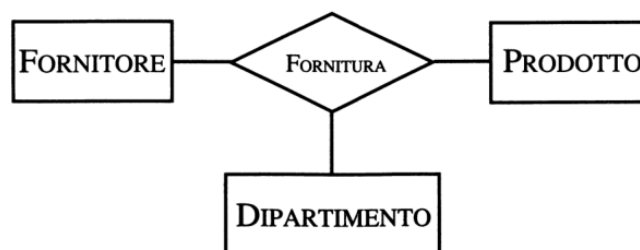


Figura 1.10: Esempio di relazione ternaria

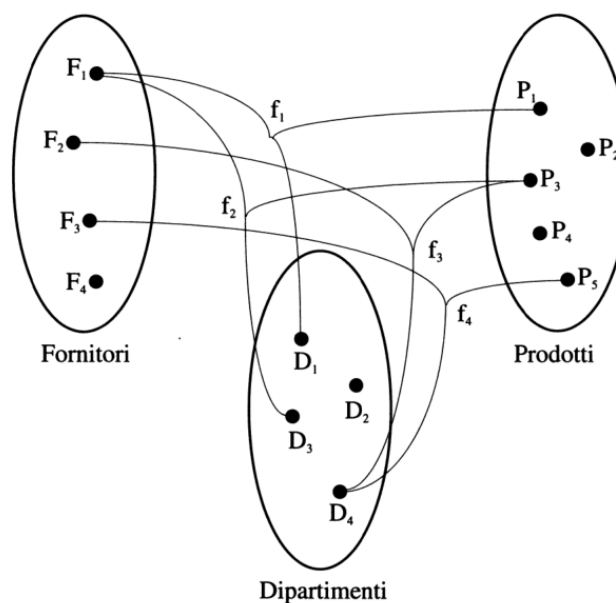


Figura 1.11: Rappresentazione estensionale dell'associazione FORNITURA

Nota che le cardinalit  nelle associazioni ternarie sono riferite alla coppia. Per esempio, assegnare la cardinalit  $(1, n)$ all'associazione di FORNITORE implica dire che ogni istanza di tale

entità è associata ad almeno una coppia `PRODOTTO-DIPARTIMENTO` e al massimo n coppie. Tale condizione si traduce nel modello estensivo in: per ogni istanza di `FORNITORE` deve esistere almeno un arco uscente.

Capitolo 2

La progettazione concettuale

2.1 Pattern di progetto

Reificazione di un attributo Abbiamo individuato un'unica entità IMPIEGATO che ha come attributi Codice, Nome, Azienda. Il problema principale dello schema è l'attributo Azienda: si tratta di una stringa associata all'entità impiegato, non possiamo controllare il campo a dovere. Per rappresentare correttamente il concetto di azienda conviene reificare l'attributo, facendolo diventare un'entità.

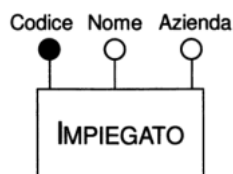


Figura 2.1: Una sola entità

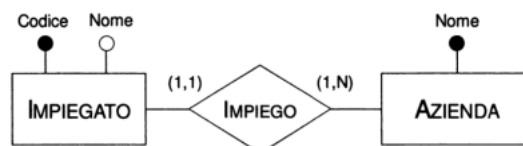


Figura 2.2: Reificazione dell'attributo Azienda

Associazione di tipo "parte di" In questi casi si vuole rappresentare che un'entità è parte di un'altra entità. Sono tipicamente relazioni uno a molti e spesso si presentano in due forme. In un caso l'entità "parte" dipende dall'esistenza di una occorrenza dell'entità che la contiene, cioè si tratta di un'entità debole (come nell'esempio di CINEMA e SALA). Nell'altro caso l'entità "parte" ha esistenza autonoma (come nell'esempio di TEAM e TECNICO).

Associazione di tipo "istanza di" In questi casi si ha che le occorrenze di un'entità della relazione sono istanze di occorrenze dell'altra entità. Nell'esempio di figura 2.4: VOLO rappresenta un concetto astratto di un volo di una compagnia aerea; VOLO REALE è l'istanza di un certo volo in un certo giorno. Il volo reale viene identificato dalla data e il volo di cui è istanza; quindi si assume che un volo non possa essere ripetuto lo stesso giorno. In

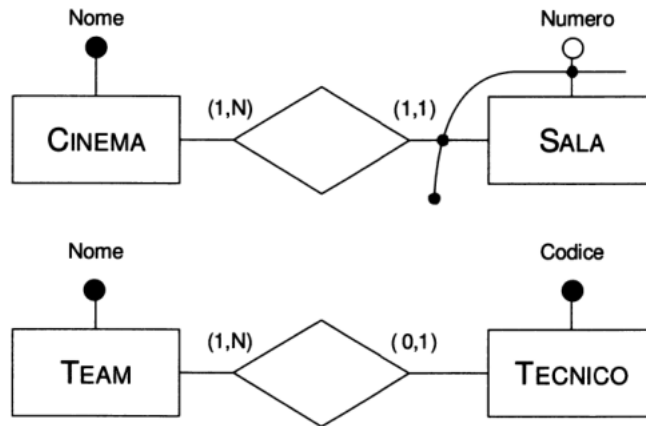


Figura 2.3: Associazione di di tipo "parte di"

questo caso, a differenza del pattern "parte di", l'entità che rappresenta l'istanza di un'altra è sempre debole.

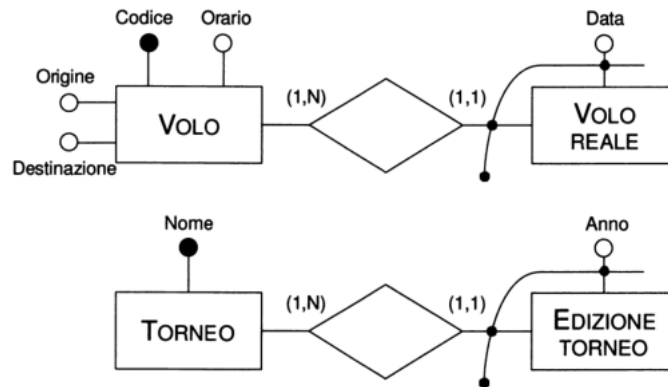


Figura 2.4: Associazione di di tipo "istanza di"

Reificazione della associazione Analizziamo il seguente esempio:

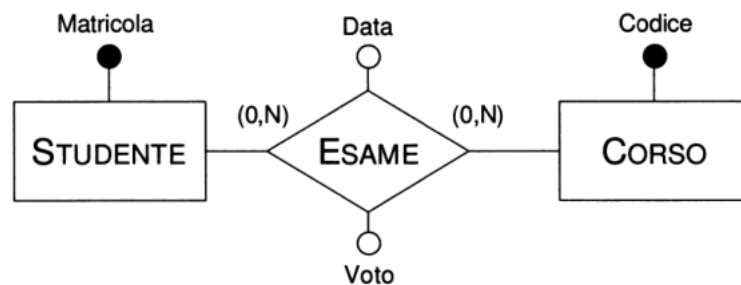


Figura 2.5: Associazione che rappresenta un concetto che lega altri concetti

la soluzione è valida solo se ogni studente può sostenere una sola volta un certo esame. Infatti le occorrenze della relazione **ESAME** sono composte dalle coppie studente-corso, senza duplicati. Se volessimo rappresentare il fatto che uno studente possa sostenere più esami allora lo schema andrebbe modificato.

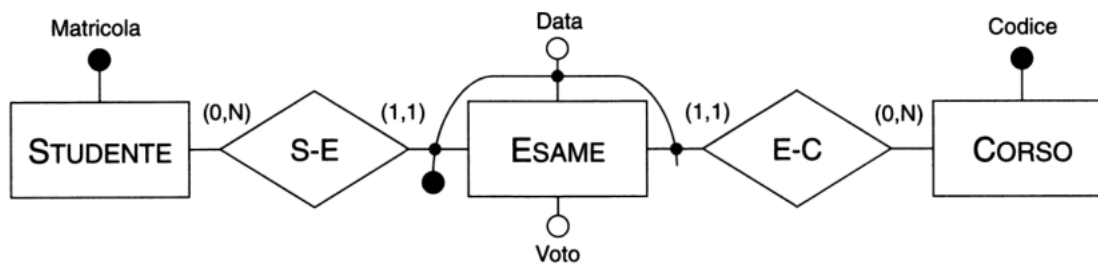


Figura 2.6: Reificazione dell'associazione ESAME

Abbiamo reificato l'associazione ESAME rappresentandola come entità (debole). L'identificazione dell'esame avviene attraverso la tripla Data, Studente, Corso.

Potremmo anche pensare di utilizzare un altro identificatore per l'esame, per esempio un codice id. In questo modo l'entità esame non sarebbe più debole. Ciò è sempre possibile ma non è la soluzione ottima: si potrebbe verificare il problema dell'inserimento ripetuto dello stesso dato nella base di dati (problema che non si riscontra nella soluzione proposta).

Reificazione dell'attributo di associazione Analizziamo il seguente esempio:

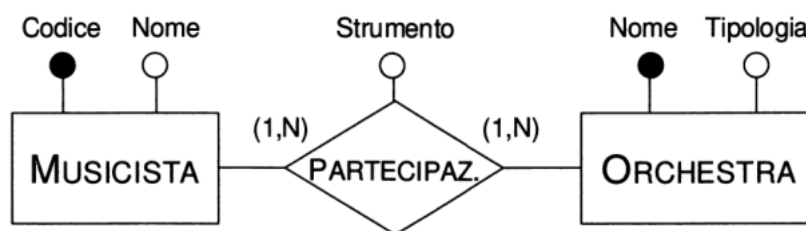


Figura 2.7: Associazione molti a molti con attributo

rappresentiamo la partecipazione di un musicista con un certo strumento ad un orchestra. Per come abbiamo disegnato lo schema, un musicista può suonare più strumenti ma suona, per ogni orchestra, sempre lo stesso strumento. Se era quello che volevamo rappresentare allora lo schema è corretto. Il difetto principale dello schema è che lo strumento viene rappresentato come una semplice stringa. Lo strumento è un concetto rilevante per l'applicazione, allora conviene reificare l'attributo dell'associazione e, per poterlo fare, dobbiamo reificare anche l'associazione. Lo schema diventa quello di figura 2.8.

Nota che in generale la trasformazione da associazione binaria ad entità debole è sempre possibile.

Storicizzazione di associazione Analizziamo l'esempio di figura 2.9.

Utilizzando lo schema riportato, per poter risalire a tutte le persone che hanno un impiego devo effettuare una ricerca sequenziale e controllare se l'attributo fine è nullo. Le prestazioni non sono ottime, conviene modificare lo schema come mostrato in figura 2.10.

In questo modo la ricerca è molto più efficiente, l'unico svantaggio è che abbiamo introdotto una nuova tabella e di conseguenza il database occuperà più spazio. Nota inoltre che nel nuovo schema abbiamo cambiato le condizioni di esistenza delle occorrenze dell'entità PERSONA. In principio volevamo inserire nel database tutte le persone che avevano un impiego (passato o presente); con il nuovo schema non possiamo esprimere tale vincolo nello schema ER, dobbiamo per forza di cose specificarlo scrivendolo da qualche parte.

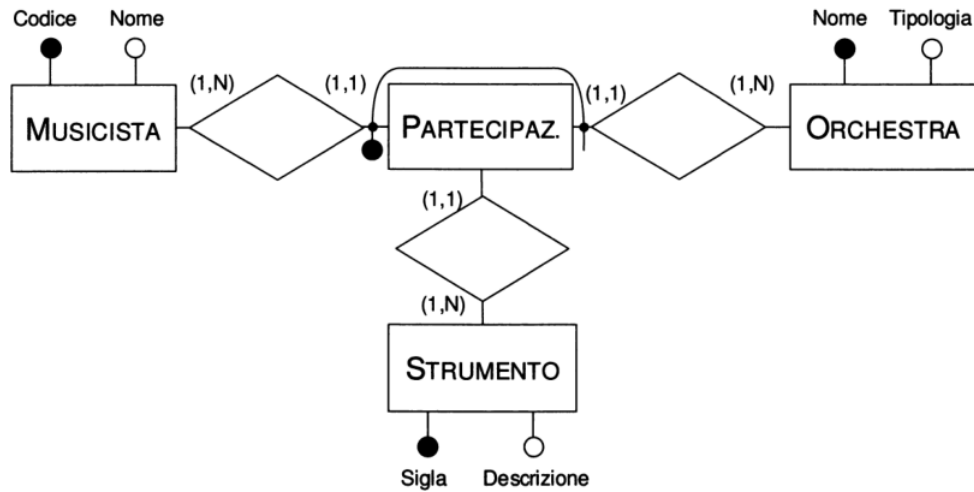


Figura 2.8: Reificazione di attributo di associazione

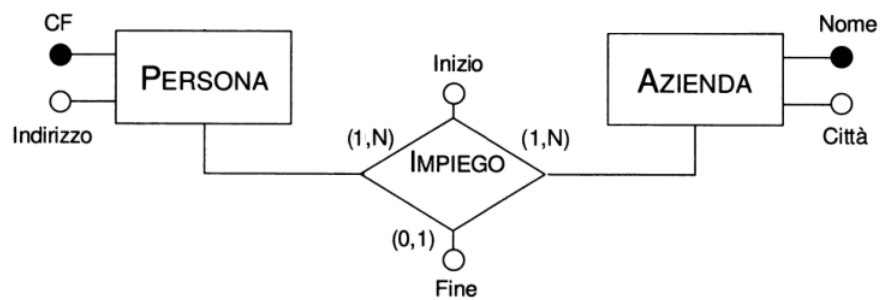


Figura 2.9

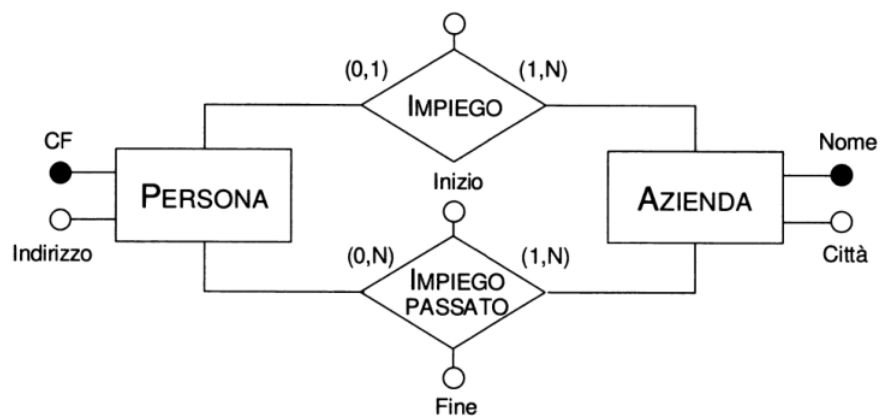


Figura 2.10: Storicizzazione di associazione

Capitolo 3

La progettazione logica

3.1 Fasi della progettazione logica

Le attività principali della progettazione logica sono la riorganizzazione dello schema concettuale e la traduzione in un modello logico.

- **Ristrutturazione dello schema ER:** è una fase indipendente dal modello logico scelto e si basa su criteri di ottimizzazione dello schema e di semplificazione della fase successiva.
- **Traduzione verso il modello logico:** fa riferimento ad uno specifico modello logico, nel nostro caso è il modello relazionale, e può includere una ulteriore ottimizzazione che si basa sulle caratteristiche del modello logico stesso.

3.2 Modello relazionale: definizioni

Relazioni matematiche e tabelle Il **prodotto cartesiano** cartesiano di due insiemi è dato dalla combinazione di tutte le coppie possibili. Per esempio:

$$\begin{aligned} \text{Nomi} &= \{\text{Mario, Antonio}\}, \text{Cognomi} = \{\text{Rossi, Bianchi, Neri}\} \\ \text{Nomi} \times \text{Cognomi} &= \{(\text{Mario, Rossi}), (\text{Mario, Bianchi}), (\text{Mario, Neri}), (\text{Antonio, Rossi}), \\ &\quad (\text{Antonio, Bianchi}), (\text{Antonio, Neri})\} \end{aligned}$$

Si nota che

$$|\text{Nomi} \times \text{Cognomi}| = |\text{Nomi}| \cdot |\text{Cognomi}|$$

Definiamo una **relazione matematica** come il sottoinsieme del prodotto cartesiano di due domini.

$$R \subset \text{Nomi} \times \text{Cognomi}$$

Per esempio

$$R = \{(\text{Antonio, Rossi}), (\text{Mario, Rossi}), (\text{Antonio, Neri})\}$$

Le relazioni possono essere rappresentate graficamente sotto forma di **tabelle**.

Schema di relazione R Uno schema di relazione è un insieme ordinato di attributi X .

$$R(X) = R(A_1, A_2, \dots, A_j, \dots, A_m), \quad X = \{A_1, A_2, \dots, A_j, \dots, A_m\}$$

R	A_1	$\dots$	A_m
$tupla\ 1 \rightarrow$	v_{11}	$\dots$	v_{1m}
$tupla\ 2 \rightarrow$	v_{21}	$\dots$	v_{2m}
$\vdots$	$\dots$	$\dots$	$\dots$
$tupla\ i \rightarrow$	v_{i1}	$\dots$	v_{im}

Istanza di relazione r Un'istanza di relazione (o semplicemente relazione) su uno schema di $R(X)$ è un insieme r di tuple su X . Spesso si usa $r(X)$ per indicare una relazione sull'insieme di attributi X , descrivendo al tempo stesso lo schema e l'istanza.

$$r = \{t_1, t_2, \dots, t_j, \dots, t_n\}$$

dove $t_i = \langle v_{i1}, v_{i2}, \dots, v_{ij}, \dots, v_{im} \rangle$ è un vettore di valori di dimensione m , in cui i valori $v_{ij} \in \text{Dominio}(A_j)$; ovvero i valori devono appartenere al dominio dell'attributo. Questi due vincoli (dimensione tupla e dominio di appartenenza valori) sono i primi due controlli che vengono effettuati dal DBMS quando si tenta di inserire una tupla in una tabella.

Vincolo di chiave Introduciamo le **superchiavi** S_K (*Super Key*):

$$S_K \subset X$$

ovvero è un sottoinsieme degli attributi. Per essere una superchiave deve valere la seguente proprietà

$$\forall t_i, t_k \in r \quad \text{si ha che} \quad t_i[S_K] \neq t_k[S_K]$$

dove $t_i[S_K] \rightarrow t_i[A_x, A_y] \rightarrow v_{ix}, v_{iy}$. Ovvero stiamo dicendo che i valori delle superchiavi tra diverse tuple appartenenti alla stessa istanza di relazione devono essere diversi.

Definiamo formalmente le **chiavi** K .

$$K \subset X$$

ovvero la chiave è un sottoinsieme degli attributi. Inoltre, una chiave

- è una superchiave;
- è minimale:

$$\forall A \in K \quad \exists t_i, t_k \in r \quad \text{tale che} \quad t_i[K \setminus A] = t_k[K \setminus A]$$

cioè stiamo dicendo che rimuovendo un solo attributo A dall'insieme chiave K , si riescono a trovare almeno due tuple appartenenti alla stessa istanza di relazione r che hanno gli stessi valori.

Abbiamo introdotto il terzo vincolo. Il terzo controllo che svolge il DBMS è proprio quello di verificare che la chiave della tupla inserita non sia già presente nella tabella. Questo terzo vincolo più i due precedenti formano sono detti **vincoli intra-relazionali**.

Vincolo di chiave esterna o vincolo di integrità referenziale Poniamo di avere $R_1(X_1)$ e $R_2(X_2)$:

$$R_1(\underline{A_1}, A_2, A_3) \quad R_2(\underline{B_1}, B_2, B_3)$$

vogliamo specificare che gli attributi A_2, A_3 devono avere gli stessi valori degli attributi B_1, B_2 . Come possiamo esprimere tale vincolo tra due tabelle? Poniamo

$$\{B_1, B_2\} = K \subset X_2 \quad \text{chiave}$$

$$\{A_2, A_3\} = F_K \subset X_1 \quad |F_K| = |K| \quad \text{chiave esterna}$$

deve valere che

$$\forall t_i \in r_1 \exists t_k \in r_2 \quad \text{t.c.} \quad [t_i[F_K] = t_k[K]] \quad \text{oppure} \quad [t_i[F_K] = \text{NULL}]$$

Abbiamo trovato il quarto vincolo degli schemi relazionali.

I 4 vincoli degli schemi relazionali Sono i vincoli che vengono gestiti dal DBMS. Questi 4 vincoli garantiscono che la base di dati si trova in uno stato consistente.

1. vincolo sulla cardinalità delle tuple;
2. vincolo sul dominio dei valori delle tuple;
3. vincolo di chiave;
4. vincolo di chiave esterna.

I primi tre formano i **vincoli intra-relazionali**, mentre l'ultimo è un vincolo **inter-relazionale**.

Definizione di base di dati Uno **schema di base di dati** è definito come segue:

$$BD = \{R_1(X_1), R_2(X_2), \dots, R_m(X_m)\} + \text{insieme di vincoli}$$

dove l'*insieme di vincoli* lo definiamo noi in fase di progettazione.

Definiamo l'**istanza di una base di dati** come:

$$bd = \{r_1, r_2, \dots, r_m\} + \text{insieme di vincoli}$$

Una base di dati si trova in uno **stato valido** se sono soddisfatti tutti i 4 vincoli. Nota che, nell'inserimento dei dati nelle tabelle, è possibile trovarsi in uno stato invalido. Ciò è dovuto ai vincoli presenti (es: ordine di inserimento). In questi casi è consentito portarsi in uno stato invalido ed effettuare una serie di operazioni sequenziali; alla fine delle operazioni, prima di procedere con l'inserimento effettivo, il DBMS controllerà che il database si trovi ancora in uno stato valido. In caso positivo l'inserimento avverrà con successo, in caso contrario il DBMS effettuerà un roll back portandosi alla situazione precedente al tentato inserimento.

3.3 Ristrutturazione di schemi ER

Le fasi di ristrutturazione di uno schema ER sono le seguenti:

- **Analisi delle ridondanze:** si decide se eliminare o mantenere eventuali ridondanze (attributi o relazioni) presenti nello schema. Solitamente si decide di togliere o mantenere una ridondanza in base alle operazioni che si intende eseguire maggiormente sulla base di dati. In generale viene fatta un'analisi sui costi in termini di prestazioni e memoria richiesta e si decide cosa fare.
- **Eliminazione delle generalizzazioni:** tutte le generalizzazioni presenti nello schema vengono analizzate e sostituite da altri costrutti. Non esiste una traduzione unica, in seguito vedremo alcune alternative che possiamo sfruttare a seconda del caso in cui ci troviamo.
- **Scelta degli identificatori principali:** si seleziona un identificatore per quelle entità che ne hanno più di uno.

3.3.1 Eliminazione delle generalizzazioni

Analizziamo lo schema ER di figura 3.1. Definiamo:

- **Generalizzazione totale:** rappresentata da una freccia nera; indica che gli elementi di E_0 appartengono a E_1 oppure a E_2 . In altre parole si ha che l'unione degli elementi di E_1 ed E_2 coincide con gli elementi di E_0 .
- **Generalizzazione parziale:** rappresentata come in figura; indica che non tutti gli elementi di E_0 appartengono a E_1 o a E_2 .
- **Generalizzazione totale/parziale disgiunta:** se l'intersezione tra E_1 ed E_2 è vuota.
- **Generalizzazione totale/parziale sovrapposta** se esistono elementi comuni tra E_1 ed E_2 .

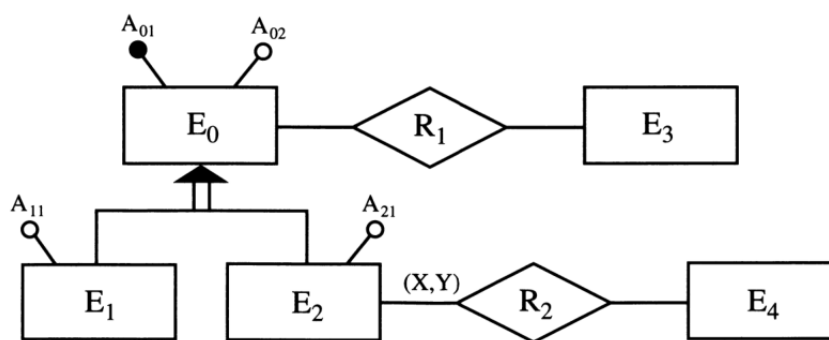


Figura 3.1: Esempio di schema ER con generalizzazione parziale

Vediamo con quali metodi di ristrutturazione possiamo andare ad eliminare la generalizzazione.

- (1) **Accorpamento delle figlie della generalizzazione al genitore.** In questo caso la classe madre eredita tutti gli attributi dei figli, i quali diventano opzionali. Inoltre viene aggiunto un nuovo attributo chiamato *Tipo*, utilizzato per indicare se l'occorrenza appartiene a una delle entità figlie. Possiamo utilizzare questa soluzione se:
 - la generalizzazione è *totale e disgiunta*: in tal caso l'attributo *Tipo* può assumere solo due valori (nel nostro esempio);
 - la generalizzazione è *totale e sovrapposta*: bisogna aggiungere un valore al dominio dell'attributo *Tipo* in modo che indichi l'appartenenza a più entità figlie;
 - la generalizzazione è *parziale e disgiunta*: in questo caso significa che esistono istanze di E_0 che hanno tutti gli attributi opzionali nulli;
 - la generalizzazione è *parziale e sovrapposta*.

Dunque possiamo utilizzare questa ristrutturazione in qualsiasi caso. Si tratta della soluzione più semplice. Da notare che in ogni caso va modificata la cardinalità con cui partecipa l'entità ottenuta nella ristrutturazione. Nello schema originale E_2 partecipava con cardinalità minima X all'associazione R_2 ; ora il minimo va messo a 0 visto che non tutte le entità figlie partecipano all'associazione.

- (2) **Accorpamento del genitore della generalizzazione nelle figlie.** In questo caso eliminiamo l'entità madre E_0 e le due entità figlie ereditano entrambe gli attributi della madre. Possiamo utilizzare questa soluzione se:

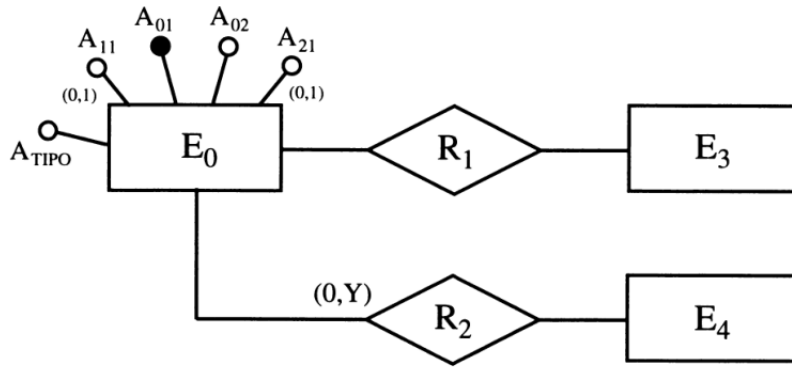


Figura 3.2: Accorpamento delle figlie della generalizzazione al genitore

- la generalizzazione è *totale e disgiunta*;
- la generalizzazione è *totale e sovrapposta*: in tal caso esistono righe uguali in tabelle diverse. Va gestito questo aspetto.

Notiamo che con questa soluzione si vengono a creare delle associazioni aggiuntive (nel nostro caso, non è detto in generale). Per quanto riguarda le cardinalità delle partecipazioni non c'è una regola generale, bisogna ragionare caso per caso basandosi sullo schema di partenza.

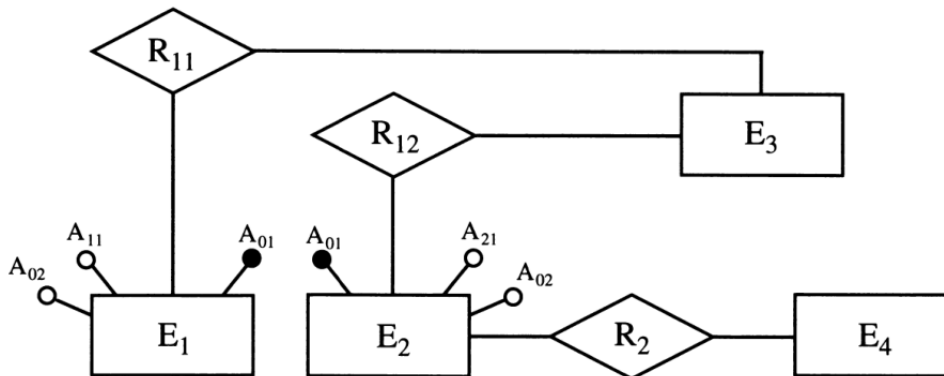


Figura 3.3: Accorpamento del genitore della generalizzazione nelle figlie

- (3) **Sostituzione della generalizzazione con associazioni.** In questo caso la generalizzazione si trasforma in due associazioni uno a uno che legano rispettivamente l'entità genitore con le entità figlie. Non ci sono trasferimenti di attributi; le entità figlie vengono identificate esternamente dall'entità genitore (diventano entità deboli). Possiamo utilizzare questa soluzione se:

- la generalizzazione è *totale e disgiunta*;
- la generalizzazione è *totale e sovrapposta*;
- la generalizzazione è *parziale e disgiunta*;
- la generalizzazione è *parziale e sovrapposta*.

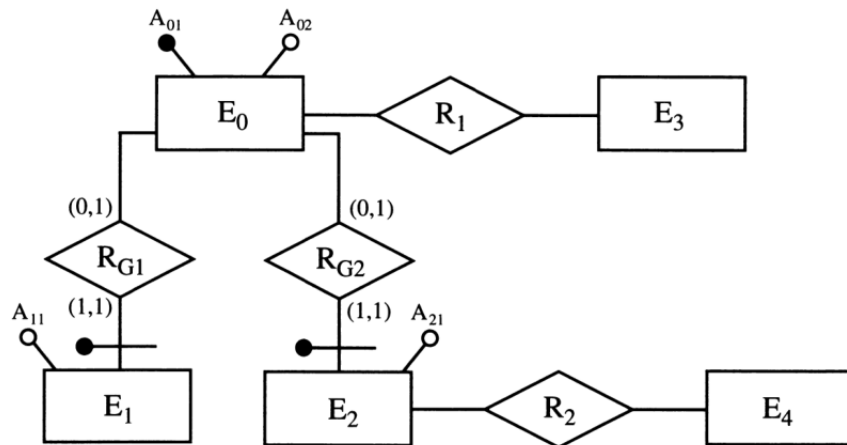


Figura 3.4: Sostituzione della generalizzazione con associazioni

Come facciamo a scegliere quale soluzione adottare?

- La soluzione (1) è valida se le operazioni più frequenti nella base di dati non fanno distinzione tra le occorrenze e tra gli attributi di E_0 , E_1 , E_2 . In questo caso le prestazioni sono ottime in quanto viene ridotto il numero degli accessi e occupa meno spazio rispetto alle altre alternative.
- Viceversa la soluzione (2) conviene utilizzarla se le operazioni più frequenti sono mirate alle specifiche istanze e attributi di E_1 , E_2 (E_0 non presente, infatti possiamo applicare la soluzione solo se la generalizzazione è totale, altrimenti ci sarebbe perdita di dati). In questo modo si ottengono prestazioni migliori ma si paga di più in termini di spazio se vengono aggiunte nuove associazioni.
- La soluzione (3) sembra la più chiara a livello di schema ER, tuttavia si porta dietro più complicazioni rispetto alle altre soluzioni a causa delle entità deboli. Alternativa valida solo quando la generalizzazione non è totale (sebbene ciò non sia necessario).

3.4 Traduzione verso il modello relazionale

Studieremo la traduzione verso il modello relazionale (tabellare). Affrontiamo il problema della traduzione caso per caso, iniziando dal caso generale (entità legate da associazioni molti a molti) che ci suggerisce l'idea generale su cui si basa la metodologia di traduzione.

Come trasformare un'entità Prendiamo come esempio un'entità E avente come attributi a_1 , a_2 , a_3 , dove a_1 è l'identificatore. Nel modello logico diventa

$$E(\underline{A_1}, A_2, A_3)$$

dove l'ordine degli attributi può essere scelto arbitrariamente e l'attributo sottolineato (cioè l'identificatore dell'entità) ora prende il nome di **chiave primaria**. Se l'entità ha più identificatori, allora bisogna decidere quale di questi è la chiave primaria. La scelta della chiave primaria non modifica la struttura che otteniamo. Se l'entità ha come identificatore un gruppo di attributi:

$$E(\underline{A_1, A_2}, A_3)$$

dunque la coppia dei due attributi forma la chiave primaria.

Come abbiamo detto, l'ordine degli attributi non conta per la struttura. L'ordine di solito viene scelto in modo che abbia un senso per noi che leggiamo la tabella. In generale il primo attributo è sempre la chiave primaria.

3.4.1 Entità e associazioni molti a molti

Facendo riferimento all'esempio riportato nella figura, otteniamo:

IMPIEGATO(Matricola, Cognome, Stipendio)
 PARTECIPAZIONE(Impiegato, Progetto, DataInizio)
 PROGETTO(Codice, Nome, Budget)



Figura 3.5: Schema ER con associazione molti a molti

Nello schema ottenuto esistono due **vincoli di integrità referenziale** tra gli attributi Impiegato e Progetto di PARTECIPAZIONE e le chiavi primarie Matricola di IMPIEGATO e Codice di PROGETTO. Il nome che abbiamo dato agli attributi in PARTECIPAZIONE è una convezione. Grazie al vincolo referenziale diciamo che i valori degli attributi che sono presenti in PARTECIPAZIONE sono gli stessi che devono essere presenti in IMPIEGATO e PROGETTO.

La traduzione da modello ER a modello logico che abbiamo analizzato vale per qualsiasi tipo di associazione molti a molti, indipendentemente dal valore della cardinalità minima di partecipazione (0, 1 oppure m).

Nel caso in cui siano presenti associazioni con più due entità partecipanti, la traduzione nel modello relazionale si effettua in maniera analoga a quella vista nel caso di associazioni binarie.

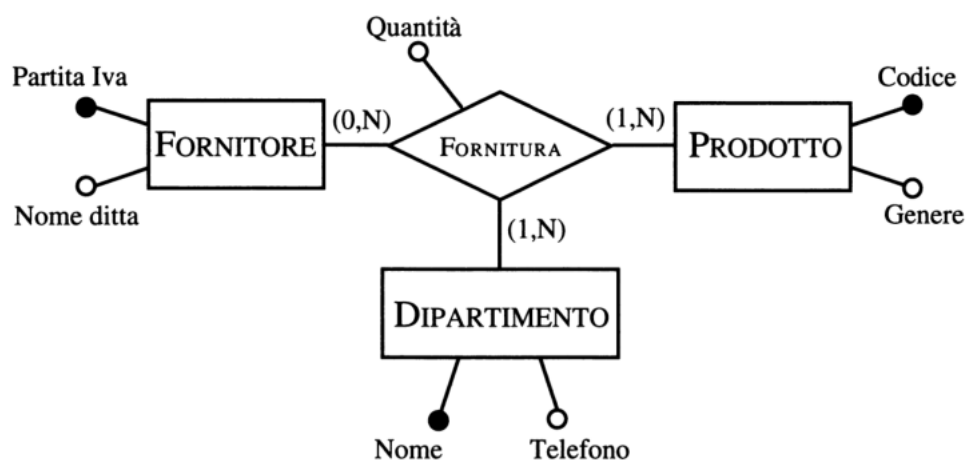


Figura 3.6: Schema ER con associazione molti a molti ternaria

FORNITORE(PartitaIVA, NomeDitta)
 PRODOTTO(Codice, Genere)
 DIPARTIMENTO(Nome, Telefono)
 FORNITURA(Fornitore, Prodotto, Dipartimento, Quantità)

La traduzione vale se la cardinalità massima di partecipazione all'associazione di tutte e tre le entità è n .

3.4.2 Associazione uno a molti

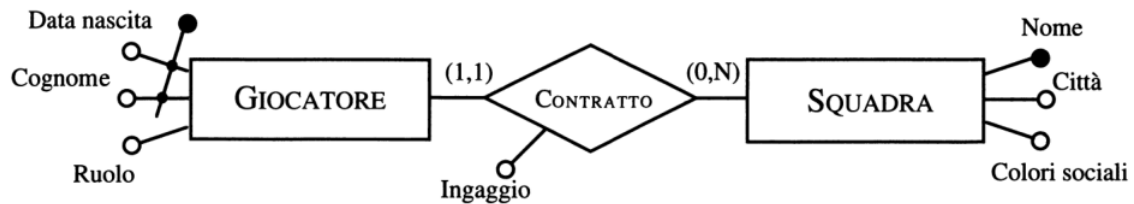


Figura 3.7: Schema ER con associazione uno a molti

Facendo riferimento all'esempio riportato nella figura, lo schema logico diventa il seguente:

SQUADRA(Nome, Città, Colori)
 GIOCATORE(Cognome, DataNascita, Ruolo, NomeSquadra, Ingaggio)

In sostanza l'entità che partecipa con cardinalità $(1,1)$ viene tradotta nel modello logico in una tabella in cui viene aggiunta la chiave primaria dell'altra tabella e gli eventuali attributi dell'associazione. Quella che abbiamo enunciato è la struttura ottima quando le entità coinvolte nell'associazione hanno cardinalità $(1,1)$ da una parte e dall'altra $(0,n)$, $(1,n)$ oppure (n,m) .

Se avessimo applicato lo stesso approccio che abbiamo visto nella sezione 3.4.1, avremmo ottenuto una soluzione non ottimale. Lo schema logico risultante sarebbe stato il seguente:

SQUADRA(Nome, Città, Colori)
 CONTRATTO(GioCatCognome, GioCatDataNascita, SquadraNome, Ingaggio)
 GIOCATORE(Cognome, DataNascita, Ruolo)

La chiave primaria dell'associazione CONTRATTO è costituita solamente dagli identificatori di GIOCATORE, infatti le cardinalità dell'associazione ci dicono che possono esistere più giocatori che giocano nella stessa squadra e che un giocatore può giocare in una singola squadra. In altre parole nella tabella CONTRATTO possono essere presenti più tuple che hanno lo stesso valore nel campo del nome della squadra. Notiamo che le relazioni GIOCATORE e CONTRATTO hanno la stessa chiave primaria e quindi conviene fonderle in un'unica relazione per evitare sprechi di spazio, si evitano inoltre alcuni controlli sui vincoli di integrità e quindi si migliorano le prestazioni.

Nel caso di una relazione ternaria in cui un'entità partecipa con cardinalità $(1,1)$ e le altre due con cardinalità massima n (minima qualsiasi), allora si procede in maniera analoga al caso binario analizzato precedentemente: si accorpa alla tabella dell'entità che partecipa con cardinalità $(1,1)$ la chiave primaria delle altre due tabelle e gli eventuali attributi dell'associazione. Nota inoltre che si ottiene lo stesso identico schema logico se si converte l'associazione ternaria descritta in una doppia associazione binaria.

3.4.3 Il caso ricorsivo

Facendo riferimento allo schema ER riportato, si ottiene il seguente schema relazionale:

PRODOTTO(Codice, Nome, Costo)
 COMPOSIZIONE(ProdottoComposto, ProdottoComponente, Quantità)

dove sussiste il vincolo di integrità referenziale tra ProdottoComposto e Codice e tra ProdottoComponente e Codice.

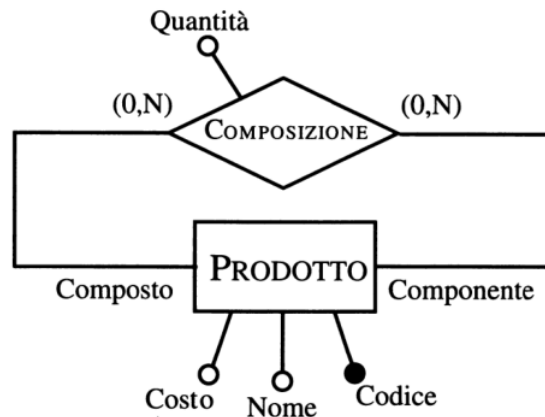


Figura 3.8: Schema ER con associazione ricorsiva

Se ponessimo la cardinalità del ramo Composto a (1, 1), cioè se tutti i prodotti fossero composti da un altro prodotto, come cambierebbe lo schema logico?

PRODOTTO(Codice, CodiceComponente, Nome, Costo, Quantità)

dove è presente il vincolo referenziale di integrità tra CodiceComponente e Codice. Nota inoltre che a causa della cardinalità (1, 1) siamo costretti ad imporre il vincolo **NOT NULL** sulla chiave esterna CodiceComponente. Tale vincolo non può essere espresso nello schema logico, va specificato e va poi imposto in fase di realizzazione in SQL.

3.4.4 Caso delle entità deboli

Le entità che hanno identificatori esterni, danno luogo a relazioni con chiavi che includono gli identificatori delle entità "identificanti".

STUDENTE(Matricola, Università, AnnoIscrizione, Cognome)
 UNIVERSITÀ(Nome, Città, Indirizzo)

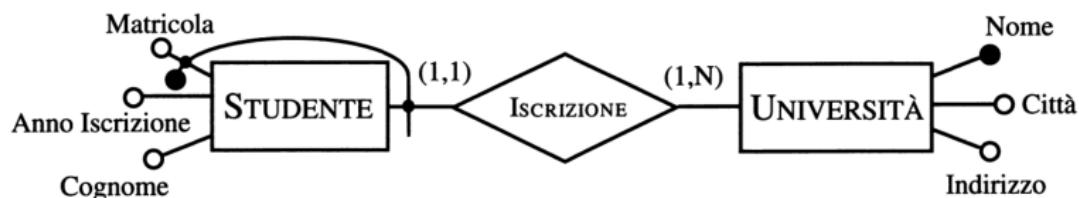


Figura 3.9: Schema ER con entità debole

dove sussiste un vincolo di integrità referenziale tra Università di STUDENTE e la chiave primaria di UNIVERSITÀ.

3.4.5 Caso in cui bisogna scegliere

Analizziamo il seguente schema ER



Figura 3.10: Schema ER con partecipazione all'associazione facoltativa

lo schema logico diventa

PROGETTO(Codice, Budget, Nome)
 IMPIEGATO(Matricola, Cognome, Stipendio, CodiceProgetto, DataInizio)

dove CodiceProgetto e Codice sono legati dal vincolo di integrità referenziale. La chiave esterna CodiceProgetto può avere valore **NULL** visto che possono esistere impiegati che non partecipano ad alcun progetto. Nota inoltre che in questo caso anche il valore della data di inizio deve essere **NULL** di conseguenza. Questo legame tra i valori degli attributi non riusciamo ad esprimerlo a livello di schema logico, dobbiamo però imporlo a livello software.

Tuttavia esiste un'altra soluzione:

PROGETTO(Codice, Budget, Nome)
 IMPIEGATO(Matricola, Cognome, Stipendio)
 PARTECIPAZIONE(Impiegato, Progetto, DataInizio)

la chiave primaria di PARTECIPAZIONE deve essere l'identificativo dell'entità che partecipa con cardinalità (0, 1). Il vantaggio di questa seconda struttura è che evitiamo di gestire i valori **NULL**. Infatti inseriamo nella tabella PARTECIPAZIONE solo gli impiegati che partecipano ad un progetto. Nella soluzione precedente, per poter trovare il numero di impiegati che partecipano ad un progetto, siamo costretti ad effettuare una ricerca sequenziale su tutte le tuple; nella seconda soluzione ci basta sapere la dimensione della tabella PARTECIPAZIONE. D'altro canto la prima soluzione risulta più efficace in termini di compattezza, spazio richiesto (2 tabelle contro 3 della seconda soluzione) e di prestazioni nell'inserimento (1 vincolo di integrità referenziale contro 2 della seconda soluzione).

Allora quale struttura preferiamo? Dobbiamo basarci sulla dimensione della base di dati (in termini di dati da inserire nelle tabelle) e in base alle operazioni che prevediamo di richiedere più spesso. Se la base di dati è piccola, allora conviene la prima soluzione; se la base di dati è grande e le operazioni di ricerca sono molto frequenti, allora è più efficace la seconda soluzione; se la base di dati è grande e le operazioni di ricerca non sono frequenti, allora conviene ancora la prima soluzione in quanto è più compatta.

Capitolo 4

Algebra e calcolo relazionale

4.1 Operazioni in algebra relazionale

L'algebra relazionale è un linguaggio procedurale basato su concetti di tipo algebrico.

4.1.1 Operazione di selezione

Definita come

$$\sigma_F(r(X)) = \{t \in r(X) \mid F(t) = \text{TRUE}\}$$

dove

- $r(X)$ è un'istanza di relazione;
- F è una qualche formula logica;
- t è una tupla.

Per quanto riguarda la cardinalità:

$$0 \leq |\sigma_F(r(X))| \leq |r(X)|$$

Esempio utilizzo: vogliamo selezionare tutti i musicisti con cognome "Neri". In algebra relazionale diventa

$$\sigma_{\text{Cognome}=\text{"Neri"}}(\text{Musicista})$$

4.1.2 Operazione di proiezione

Definita come

$$\pi_Y(r(X)) = \{t[Y] \in r(X)\}$$

dove

- $Y \subseteq X$;
- $t[Y]$ è una tupla ridotta sui valori di Y .

Per quanto riguarda la cardinalità:

$$|\pi_Y(r(X))| \leq |r(X)|$$

l'uguaglianza vale se Y è superchiave (non per forza minimale).

Esempio utilizzo: vogliamo selezionare tutti i musicisti che hanno cognome "Neri", di questi vogliamo sapere il codice fiscale, nome e cognome.

$$\pi_{(\text{CF}, \text{Nome}, \text{Cognome})}(\sigma_{\text{Cognome}=\text{"Neri"}}(\text{Musicista})) \text{ in combinazione con una selezione}$$

4.1.3 Operazione di ridenominazione

Dato uno schema relazionale $R(A_1, \dots, A_m)$, possiamo definire tre casi di ridenominazione:

- la ridenominazione del nome dello schema e del nome degli attributi

$$\rho_{S(B_1, \dots, B_m)}(R(A_1, \dots, A_m))$$

per esempio:

$$\rho_{\text{ART}(\text{CF}, \text{Cognome})}(\text{ARTISTI}(\text{CodFis}, \text{Cog}))$$

- la ridenominazione del nome dello schema

$$\rho_S(R(A_1, \dots, A_m))$$

ad esempio:

$\rho_{\text{ART}}(\text{ARTISTI}(\text{CodFis}, \text{Cog}))$ applicato ad uno schema con nome
 $\rho_{\text{ART}}(\pi_{\text{CF}, \text{Cognome}}(\text{Musicisti}))$ applicato al risultato di un'operazione

- la ridenominazione dei soli attributi

$$\rho_{(B_1, \dots, B_m)}(R(A_1, \dots, A_m))$$

Nei casi in cui dobbiamo scrivere un'espressione complessa, è utile usare l'operatore di assegnamento per dare un nome al risultato ottenuto. Per esempio:

$$\text{MUS} \leftarrow \rho_{\text{AnnoNascita} \leq 1980}(\text{MUSICISTA})$$

Si tratta di una sorta di assegnazione di un valore ad una variabile.

4.1.4 Operazioni insiemistiche: unione, intersezione e differenza

Date due istanze di relazione $r_1(X), r_2(Y)$, definiamo

- **Unione**

$$r_1(X) \cup r_2(Y) = \{t \mid t \in r_1(X) \text{ OR } t \in r_2(Y)\}$$

- **Intersezione**

$$r_1(X) \cap r_2(Y) = \{t \mid t \in r_1(X) \text{ AND } t \in r_2(Y)\}$$

- **Differenza**

$$r_1(X) \setminus r_2(Y) = \{t \mid t \in r_1(X) \text{ AND } t \notin r_2(Y)\}$$

Tali operazioni insiemistiche sono valide se

1. $|X| = |Y|$, cioè gli insiemi degli attributi hanno stessa cardinalità;
2. $\text{dom}(A_i) = \text{dom}(B_i) \quad \forall i = 1, \dots, |X|$, avendo che $X = \{A_i\}, Y = \{B_i\}$;
3. i nomi degli attributi tra i due schemi di relazione sono uguali. Se non sono uguali dobbiamo utilizzare la *ridenominazione* prima di utilizzare le operazioni insiemistiche.

4.1.5 Operazione di correlazione: Join

Esistono due tipologie di *join*:

- **Join naturale.** Operazione indicata con il simbolo $\bowtie$; mette in correlazione dati di due tabelle diverse in base a valori uguali su attributi che hanno lo stesso nome (e che sono compatibili).

$$r_1(X) \bowtie r_2(Y) = \{t \mid t \in r(X \cup Y), t[X] \in r_1(X) \text{ AND } t[Y] \in r_2(Y)\}$$

Per poter utilizzare il join naturale senza dovere ogni volta utilizzare la ridenominazione, dobbiamo progettare i nomi correttamente a livello di schema relazionale.

- **Theta join.** A differenza del join naturale, il theta join non richiede che le due tabelle abbiano almeno un attributo con nome uguale. Formalmente viene definito come segue

$$r_1(X) \bowtie_F r_2(Y) = \sigma_F(r_1(X) \times r_2(Y))$$

In generale il theta join restituisce sempre più colonne rispetto a quelle che restituisce il natural join. Infatti quest'ultimo rimuove la colonna comune alle due tabelle (la colonna che ha lo stesso nome), mentre il theta join ritorna una tabella avente un numero di colonne che è pari alla somma delle colonne delle due di partenza. Per poter rendere un'operazione di theta join equivalente a una di natural join, dobbiamo applicare una proiezione che elimina gli attributi in comune alle due tabelle. Per esempio

$$\begin{aligned} & \text{MUSICISTA}(\underline{\text{CF}}, \text{Nome}, \text{Cognome}, \text{AnnoNascita}) \\ & \text{SUONA}(\text{Musicista}, \text{Strumento}) \\ & \pi_{\text{CF}, \text{Nome}, \text{Cognome}, \text{Strumento}}(\text{MUSICISTA} \bowtie_{\text{CF}=\text{Musicista}} \text{SUONA}) \end{aligned}$$

JOIN incompleto e JOIN esterno Il join è completo se tutte le tuple delle due tabelle coinvolte partecipano al risultato. Altrimenti viene detto **incompleto**. In generale, un join incompleto è la conseguenza di un join fatto su attributi che non sono chiave e/o sui quali non è presente un vincolo di integrità referenziale.

In alcuni casi si è interessati a fare apparire nel risultato anche le tuple che non verificano la condizione di join. In tal caso si parla di join **esterno**. Un join esterno può essere di tre tipi:

- *left join*: si mantengono tutte le tuple della tabella sinistra del join. Si indica come segue:

$$R_1(X) \Joinleft R_2(X)$$

- *right join*: si mantengono tutte le tuple della tabella destra del join. Si indica come segue:

$$R_1(X) \Joinright R_2(X)$$

- *outer join*: si mantengono le tuple di entrambe le tabelle. Si indica come segue:

$$R_1(X) \Joinfull R_2(X)$$

4.1.6 Operazioni di raggruppamento

Introduciamo la **proiezione generalizzata**: permette di aggiungere alla lista degli attributi delle funzioni sugli attributi.

$$\pi_{\text{Data}, \text{Teatro}, \text{Prezzo} * \text{Biglietti}}(\text{MUSICISTA})$$

La colonna generata dall'operazione sugli attributi non ha un nome. Per attribuire un nome si usa la ridenominazione.

$$\rho(\text{Data}, \text{Teatro}, \text{Ricavo})(\pi_{\text{Data}, \text{Teatro}, \text{Prezzo} * \text{Biglietti}}(\text{MUSICISTA}))$$

Le **operazioni di aggregazione** permettono di fare dei calcoli sugli attributi di una relazione. Le operazioni che possiamo effettuare sono **SUM**, **MAX**, **MIN**, **AVG**, **COUNT**. La notazione utilizzata per tali operazioni è la seguente:

$$F_{f_1(A_1), f_2(A_1), f_3(A_2, A_3), \dots}(R(X))$$

dove F indica che stiamo applicando una o più funzioni di aggregazione. Il risultato è una relazione che ha un numero di attributi pari al numero di funzioni calcolate e una cardinalità pari a 1 (tupla di risultati delle operazioni). Per esempio:

$$\begin{aligned} \text{MUS} &\leftarrow \pi_{\text{Data}, \text{Teatro}, \text{Prezzo} * \text{Biglietti}}(\text{MUSICISTA}) \\ F_{\text{SUM}(\text{Ricavo}), \text{AVG}(\text{Ricavo})}(\rho(\text{Data}, \text{Teatro}, \text{Ricavo})(\text{MUS})) \end{aligned}$$

Un'altra operazione molto utile è il **raggruppamento**. L'operazione di raggruppamento permette di calcolare le funzioni di aggregazione su gruppi di tuple definite su un insieme degli attributi della relazione invece che su tutta la relazione.

$$(A_i, A_j, \dots) F_{f_1(A_1), f_2(A_1), f_3(A_2), \dots}(R(X))$$

dove $A_i, A_j \in X$ sono attributi di $R(X)$. Prima vengono effettuate le operazioni di raggruppamento, poi le operazioni di aggregazione sui gruppi ottenuti. La relazione calcolata avrà grado pari alla somma della lista degli attributi di raggruppamento e della lista delle funzioni calcolate. La cardinalità della relazione sarà pari al numero di gruppi creati (dipende quindi dal numero di valori distinti sugli attributi di raggruppamento). In SQL:

```
1 SELECT A7, SUM(A5)
2 FROM R
3 GROUP BY A7;
```

viene aggiunto al **SELECT** l'attributo di raggruppamento; nel **SELECT** non vanno aggiunti altri attributi non presenti nel raggruppamento altrimenti viene generato un errore.

4.2 Algebra relazionale e valori nulli

La presenza di un valore **NULL** nel campo di una tupla può significare che:

- il valore è sconosciuto: sappiamo che esiste ma non lo sappiamo al momento dell'insierimento dei dati;
- non è disponibile: dati che non possiamo avere (es: riservatezza);
- l'attributo è opzionale.

Per trattare i valori nulli si utilizza la logica a tre valori. Oltre ai valori *vero*, *falso*, viene aggiunto il valore *sconosciuto*.

AND	T	F	U	OR	T	F	U	NOT	
	T	T	F	U	T	T	T	T	F
	F	F	F	F	F	T	F	F	T
	U	U	F	U	U	T	U	U	U

In algebra relazionale il valore **NULL** viene trattato come un valore falso. Per verificare se un attributo $A \in X$ di una relazione $r(X)$ è nullo oppure no, si usa la notazione $A \text{ IS NULL}$. Questa restituirà vero se la tupla $t \in r(x)$ ha un valore nullo sull'attributo A , falso altrimenti.

L'operazione di selezione, in presenza di valori nulli, scarta le operazioni che restituiscono U (sconosciuto). Se vogliamo includere nel risultato solo le tuple che hanno il valore nullo in un certo attributo dobbiamo usare la seguente notazione:

$$\sigma_{A_3 \text{ IS NULL}}(R(X))$$

Per quanto riguarda l'operazione di proiezione, potrebbe presentarsi la situazione $\text{NULL} = \text{NULL}$ che restituisce falso e dunque potremmo avere come risultato dell'operazione di selezione una sfilza di **NULL**. Per esempio:

$$\pi_{A_3}(R(X))$$

il risultato dell'interrogazione in SQL potrebbe ritornare

A_3
NULL
NULL
44
34

oppure

A_3
44
34

il risultato dell'operazione dipende dal DBMS utilizzato, ognuno decide come gestire i valori nulli.

Capitolo 5

Normalizzazione

5.1 Introduzione

Solitamente viene applicata nei casi in cui abbiamo a disposizione lo schema logico o l'implementazione di un database ma non abbiamo lo schema concettuale. In generale in questi casi conviene ricavarsi lo schema ER partendo dallo schema relazionale; si apportano modifiche necessarie al fine di rimuovere le ripetizioni e si cerca di migliorare lo schema. Alla fine si ricostruisce lo schema relazionale partendo dallo schema ER ricavato.

Le tecniche di normalizzazione ci permettono di sistemare lo schema logico senza andare a ricavarci lo schema ER.

Partendo da uno schema relazionale fornito, si procede a verificare se rispetta i requisiti delle forme normali.

Definizione 5.1 (Dipendenza funzionale). *Una dipendenza funzionale di uno schema relazionale $R(Z)$ è definita come*

$$X \rightarrow Y$$

dove $X \subset Z, Y \subset Z, X \cap Y = \emptyset$. Impone una dipendenza tra due sottoinsiemi degli attributi di uno schema relazionale.

Se $X \rightarrow Y$ significa che, se conosco i valori degli attributi contenuti in X di una tupla, allora conosco di conseguenza anche i valori sugli attributi contenuti in Y della stessa.

5.2 La prima forma normale

La prima forma normale richiede che tutti gli attributi abbiano dominio atomico, ovvero devono essere semplici.

5.3 La seconda forma normale

Definizione 5.2 (Attributo primo). *Un attributo si dice primo se fa parte di una chiave.*

La seconda forma normale richiede che tutti gli attributi non primi dipendano in maniera funzionale completa dalla chiave primaria intera.

5.3.1 Terza forma normale e forma normale di Boyce e Codd

La terza forma normale richiede che in tutte le dipendenze funzionali dello schema di relazione R , X sia superchiave oppure che Y sia attributo primo ¹.

¹Con X e Y si intendono i due sottoinsiemi degli attributi che compaiono nella definizione di dipendenza funzionale ($X \rightarrow Y$).

Progettando una base di dati partendo dallo modello concettuale e poi ricavando di conseguenza il modello logico, è raro che in una dipendenza funzionale Y sia un attributo primo. Per questo motivo viene introdotta una terza forma normale più restrittiva, questa prende il nome di forma normale di Boyce e Codd.

La forma normale di Boyce e Codd richiede che in tutte le dipendenze funzionali dello schema di relazione R , X sia superchiave.

Capitolo 6

Generalità sul linguaggio SQL

6.1 La definizione delle tabelle

Il linguaggio SQL possiede i comandi per creare, modificare ed eliminare le tabelle dal database relazionale, cioè le funzioni di linguaggio DDL (Data Definition Language):

- creazione ed rimozione di una tabella

```
1 CREATE TABLE table_name (  
2     column1 datatype,  
3     column2 datatype,  
4     column3 datatype,  
5     ...  
6 );
```

```
1 DROP TABLE table_name;
```

- aggiungere o togliere una colonna

```
1 ALTER TABLE table_name  
2 ADD column_name datatype;
```

```
1 ALTER TABLE table_name  
2 DROP COLUMN column_name;
```

6.2 I comandi per la manipolazione dei dati

Il linguaggio SQL possiede i comandi per inserire, modificare ed eliminare le righe di una tabella, cioè le funzioni di linguaggio DML (Data Manipulation Language):

- inserire una nuova riga nella tabella

```
1 INSERT INTO table_name (column1, column2, column3, ...)  
2 VALUES (value1, value2, value3, ...);
```

- aggiornare i valori nella tabella

```
1 UPDATE table_name  
2 SET column1 = value1, column2 = value2, ...
```

```
3 WHERE condition;
```

- cancellare le righe dalla tabella

```
1 DELETE FROM table_name WHERE condition;
```

6.3 Il comando SELECT

Il comando **SELECT** è il comando principale di SQL che realizza le funzioni di linguaggio per le interrogazioni (Query Language).

```
1 SELECT column1, column2, ... (*)
2 FROM table_name1, table_name2, ...
3 WHERE condition AND/OR/NOT condition
```

6.4 Le operazioni relazionali nel linguaggio SQL

6.4.1 Operazione relazionale di selezione

Operazione che permette di selezionare tutte le colonne di una tabella e che torna solo le righe che soddisfano la condizione.

```
1 SELECT *
2 FROM table_name
3 WHERE condition
```

Per esempio:

```
1 Select *
2 From Anagrafica
3 Where Indirizzo = 'Milano'
```

torna tutte le righe della tabella che hanno Milano come indirizzo.

6.4.2 Operazione relazionale di proiezione

Operazione che torna tutti i valori delle delle colonne specificate.

```
1 SELECT column1, column2, ...
2 FROM table_name
```

Per esempio:

```
1 Select Codice, Nome
2 From Anagrafica
```

torna l'elenco dei codici e dei nomi delle anagrafiche .

6.4.3 Operazione relazionale di congiunzione

```

1 SELECT column1, column2, ...
2 FROM table_name1, table_name2, ...
3 WHERE table_name1.attrib1 = table_name2.attrib2
4 AND condizione

```

Per esempio:

```

1 Select Nome, Importo
2 From Anagrafica, Movimento
3 Where Anagrafica.Codice= Movimento.Codice
4 And Indirizzo = 'Milano'

```

6.5 Le funzioni di aggregazione

6.5.1 Count

```

1 SELECT COUNT(*)
2 FROM table_name
3 WHERE condition;

```

6.5.2 Sum

Restituisce la somma di tutti i valori contenuti in una colonna specificata (l'attributo utilizzato nel calcolo deve essere di tipo numerico).

```

1 SELECT SUM(column_name)
2 FROM table_name
3 WHERE condition;

```

6.5.3 Avg

Calcola la media dei valori (numerici) contenuti in una determinata colonna di una tabella.

```

1 SELECT COUNT(column_name)
2 FROM table_name
3 WHERE condition;

```

6.5.4 Min e max

```

1 SELECT MIN|MAX(column_name)
2 FROM table_name
3 WHERE condition;

```

6.6 Ordinamento

```

1 SELECT column1, column2, ...
2 FROM table_name

```

```
3 ORDER BY column1, column2, ... ASC|DESC;
```

L'ordinamento crescente è di default.

6.7 Raggruppamenti

Serve per raggruppare un insieme di righe aventi lo stesso valore nelle colonne indicate: produce una riga di risultati per ogni raggruppamento. Viene usata con le funzioni di aggregazione: per ciascuna riga della tabella risultante viene prodotto un valore di raggruppamento.

```
1 SELECT column_name(s)
2 FROM table_name
3 WHERE condition
4 GROUP BY column_name(s)
5 ORDER BY column_name(s);
```

Per esempio:

```
1 SELECT COUNT(CustomerID), Country
2 FROM Customers
3 GROUP BY Country
4 ORDER BY COUNT(CustomerID) DESC;
```

6.8 Condizioni di ricerca

6.8.1 Between

Controlla se un valore è compreso all'interno di un intervallo di valori, inclusi gli estremi.

```
1 SELECT column_name(s)
2 FROM table_name
3 WHERE column_name BETWEEN value1 AND value2;
```

6.8.2 In

Controlla le righe che hanno i valori di un attributo compresi in una lista di valori indicati.

```
1 SELECT column_name(s)
2 FROM table_name
3 WHERE column_name IN (value1, value2, ...);
```

6.8.3 Like

Utilizzato per ricercare un particolare pattern in una colonna.

- % rappresenta zero, uno o più caratteri;
- _ rappresenta un singolo carattere.

```
1 SELECT column1, column2, ...
2 FROM table_name
3 WHERE columnN LIKE pattern;
```

Per esempio:

```
1 SELECT * FROM Customers
2 WHERE CustomerName LIKE 'a%';
```

seleziona tutti i clienti che iniziano con la lettera 'a'.